

MC-25 Compiler - Peephole Optimization Report

1. Introduction

This report presents the peephole optimization implemented for the MC-25 compiler as part of Team Project 2. The objective of the optimization is to reduce unnecessary MIPS instructions generated by the compiler while preserving the correctness of the generated program.

The optimization was implemented in the compiler's `codegen.c` file. The generated MIPS assembly was tested using MARS 4.5, and Instruction Statistics were used to compare the unoptimized and optimized versions.

2. Objective

The main objective of the optimization is to identify and remove redundant MIPS instructions that do not contribute to the final program result.

Two redundant instruction patterns were targeted:

- `add $t0, $t0, $zero`, which leaves the value of `$t0` unchanged.
- A redundant load immediately following a store to the same memory location, such as:

```
sw $t0, x
lw $t0, x
```

When the value stored in `x` is already available in `$t0`, loading it immediately back into `$t0` is unnecessary.

Removing these instructions reduces the number of instructions executed while maintaining the same program output.

3. Peephole Optimization Implementation

The peephole optimization was implemented in `codegen.c` through the `peephole_optimize()` function.

The optimization examines the generated MIPS instruction sequence and removes recognized redundant instruction patterns. The optimization is applied before the generated assembly is written to the output file.

The following redundant instruction was removed:

```
add $t0, $t0, $zero
```

The optimizer also identifies the following store-load sequence:

```
sw $t0, x
lw $t0, x
```

and removes the unnecessary `lw` instruction.

The optimized instruction sequence therefore performs the same computation with fewer executed instructions.

4. Unoptimized and Optimized Code

4.1 Unoptimized Code

Before applying the peephole optimization, the generated code contained redundant instructions:

```
main:
    li $t0, 5
    sw $t0, x
    lw $t0, x
    addiu $sp, $sp, -8
    sw $t0, 4($sp)
    li $t0, 10
    addiu $sp, $sp, -8
    sw $t0, 4($sp)
    li $t0, 2
    lw $t1, 4($sp)
    addiu $sp, $sp, 8
    mul $t0, $t1, $t0
    lw $t1, 4($sp)
    addiu $sp, $sp, 8
    add $t0, $t1, $t0
    add $t0, $t0, $zero
    move $a0, $t0
    li $v0, 1
    syscall
    li $v0, 10
    syscall
```

Two examples of redundant operations are visible in this version:

```
lw $t0, x
```

and:

```
add $t0, $t0, $zero
```

4.2 Optimized Code

After applying the peephole optimization, the redundant instructions were removed:

```
main:
    li $t0, 5
    sw $t0, x
    addiu $sp, $sp, -8
    sw $t0, 4($sp)
    li $t0, 10
    addiu $sp, $sp, -8
    sw $t0, 4($sp)
    li $t0, 2
    lw $t1, 4($sp)
    addiu $sp, $sp, 8
    mul $t0, $t1, $t0
    lw $t1, 4($sp)
    addiu $sp, $sp, 8
    add $t0, $t1, $t0
    move $a0, $t0
    li $v0, 1
    syscall
    li $v0, 10
    syscall
```

The optimized version removes the unnecessary load and the redundant addition while preserving the program's computation.

5. MARS Instruction Statistics

The two versions were executed using MARS 4.5 and analyzed using the MARS Instruction Statistics tool.

Version	Total Instructions	Program Output
Unoptimized	23	25
Optimized	20	25
Reduction	3	No change

The optimization reduced the number of executed instructions from **23 to 20**.

The reduction is:

$$23 - 20 = 3 \text{ instructions}$$

The percentage reduction is:

$$\frac{23 - 20}{23} \times 100 = 13.04\%$$

Therefore, the optimized version achieved an approximately **13.04% reduction in executed instructions**.

Most importantly, both versions produced the same output:

25

This demonstrates that the optimization preserved the functional behavior of the program while reducing unnecessary instructions.

5.1 Profiling Result

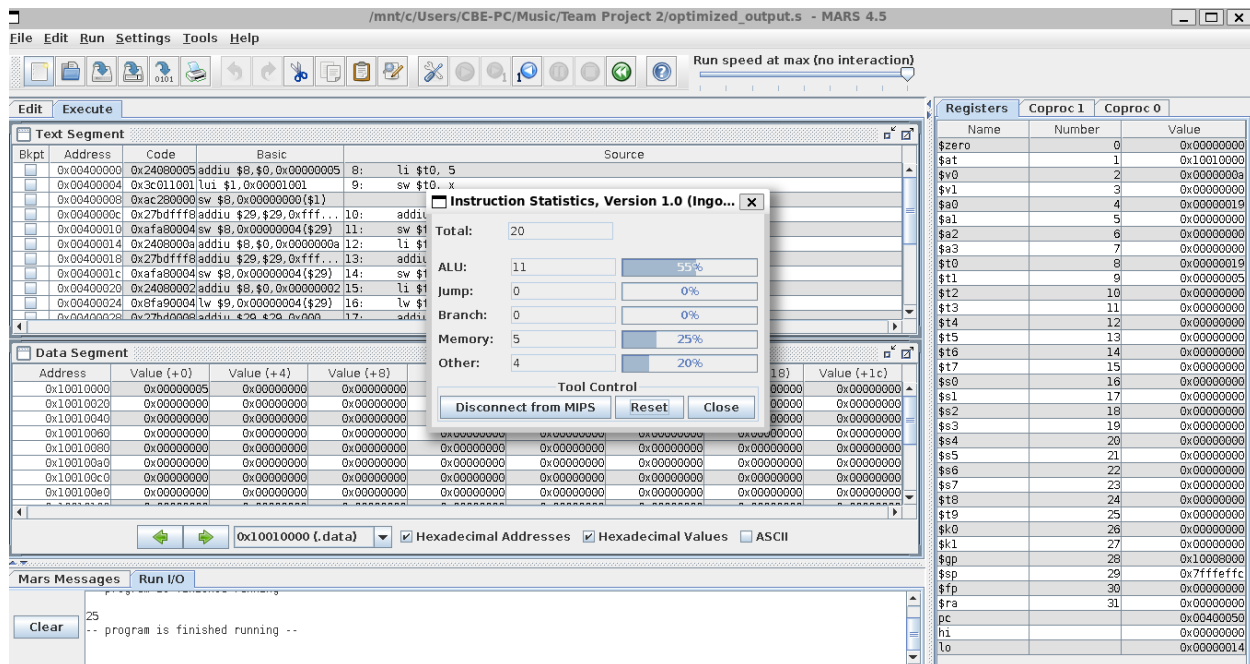
The MARS profiling results can be summarized as follows:

- **Unoptimized:** 23 instructions
- **Optimized:** 20 instructions
- **Instructions removed:** 3
- **Instruction reduction:** 13.04%
- **Unoptimized output:** 25
- **Optimized output:** 25
- **Functional behavior:** Preserved

The results demonstrate that the peephole optimization successfully reduces instruction execution without changing the program's result.

The screenshot shows the MARS 4.5 MIPS simulator interface. The main window displays assembly code with instructions like 'li \$t0, 5' and 'sw \$t0, x'. An 'Instruction Statistics' dialog box is open, showing a total of 23 instructions, with a breakdown: ALU (13, 57%), Memory (6, 26%), Other (4, 17%), Branch (0, 0%), and Jump (0, 0%). The 'Registers' panel on the right shows the state of registers \$zero through \$lo. The 'Mars Messages' panel at the bottom shows the message 'program is finished running --'.

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x10010000
\$v0	2	0x0000000a
\$v1	3	0x00000000
\$a0	4	0x00000019
\$a1	5	0x00000000
\$a2	6	0x00000000
\$a3	7	0x00000000
\$t0	8	0x00000019
\$t1	9	0x00000005
\$t2	10	0x00000000
\$t3	11	0x00000000
\$t4	12	0x00000000
\$t5	13	0x00000000
\$t6	14	0x00000000
\$t7	15	0x00000000
\$s0	16	0x00000000
\$s1	17	0x00000000
\$s2	18	0x00000000
\$s3	19	0x00000000
\$s4	20	0x00000000
\$s5	21	0x00000000
\$s6	22	0x00000000
\$s7	23	0x00000000
\$t8	24	0x00000000
\$t9	25	0x00000000
\$k0	26	0x00000000
\$k1	27	0x00000000
\$gp	28	0x10000000
\$sp	29	0x7ffffcfc
\$fp	30	0x00000000
\$ra	31	0x00000000
pc		0x0040005c
hi		0x00000000
lo		0x00000014



6. Conclusion

The MC-25 compiler's peephole optimization successfully removes redundant MIPS instructions generated during code generation. The optimization was implemented in `codegen.c` and tested using MARS 4.5.

MARS Instruction Statistics showed a reduction from **23 instructions to 20 instructions**, corresponding to an approximately **13.04% reduction in executed instructions**. At the same time, both the unoptimized and optimized programs produced the same output of 25.

These results demonstrate that the optimization improves the generated code by eliminating unnecessary instructions while preserving program correctness.

The optimization therefore satisfies the peephole optimization and MARS profiling objectives of Team Project 2.